

Constraint-Aware DeepCAD 实现文档

1. 项目定位

基于原生 DeepCAD 架构，**不新增数据集、不人工标注**，通过：

- 自动几何约束挖掘
- 约束感知 Transformer
- 约束一致性损失

实现带工程约束的 CAD 命令序列生成，缓解原生 DeepCAD **无约束、拓扑不合法、工程难落地** 等问题。

2. 核心目标

- 从 DeepCAD 原始命令序列自动提取约束（水平 / 竖直 / 平行 / 垂直 / 共线）
- 改造 DeepCAD Transformer，支持约束引导生成
- 加入约束损失，提升约束满足率与拓扑合法性
- 完全复用 DeepCAD 官方数据集，训练 / 测试流程不变

3. 支持的约束类型（仅 2D 草图，计算最简单）

中文 英文

水平 Horizontal

竖直 Vertical

平行 Parallel

垂直 Perpendicular

共线 Collinear

4. 整体流程（三步）

1. 离线约束提取（一次性）

执行 CAD 命令 → 几何实体 → 几何计算 → **约束图**

2. 模型改造

DeepCAD Transformer + 约束编码 + 约束交叉注意力 + 约束预测头

3. 训练与评估

原生损失 + 约束一致性损失；评估：约束满足率、几何误差、拓扑合法性

5. 约束提取算法（自动，无标注）

5.1 输入 / 输出

- **输入**：DeepCAD JSON 命令序列
- **输出**：约束字典（见下）

5.2 约束字典结构

```
{  
  "horizontal": [线ID列表],  
  "vertical": [线ID列表],  
  "parallel": [(线A, 线B)列表],  
  "perpendicular": [(线A, 线B)列表],  
  "collinear": [(线A, 线B)列表]  
}
```

5.3 几何判断规则（统一阈值）

```
# 阈值（论文标准）  
ANGLE_THRESH = 2.0          # 角度误差 < 2°  
DIST_THRESH = 1e-3          # 距离误差 < 0.001  
EPS = 1e-5  
  
# 线段表示  
line = {  
  "id": int,  
  "start": (x, y, z),  
  "end": (x, y, z),  
  "vec": (dx, dy, dz)  
}
```

判定：

1. **水平**： $\text{abs}(\text{line.vec.z}) < \text{EPS}$
2. **竖直**： $\text{abs}(\text{line.vec.x}) < \text{EPS}$ and $\text{abs}(\text{line.vec.y}) < \text{EPS}$
3. **平行**：

```
cosθ = dot(v1, v2) / (|v1| * |v2|)
θ = arccos(abs(cosθ))
θ < ANGLE_THRESH
```

4. 垂直:

```
cosθ = dot(v1, v2) / (|v1| * |v2|)
θ = arccos(abs(cosθ))
abs(θ - 90°) < ANGLE_THRESH
```

5. 共线:

平行 + 点 p3 到直线 L1 距离 < DIST_THRESH
距离公式: $|(p2 - p1) \times (p3 - p1)| / |p2 - p1|$

6. 模型结构修改 (最小侵入式)

基于 DeepCAD 原有 Transformer。

6.1 新增模块

- **Constraint Embedding**
约束类型编码: HORIZONTAL, VERTICAL, PARALLEL, PERPENDICULAR, COLLINEAR, NONE
- **Constraint-Aware Cross-Attention**
命令序列与约束图交叉注意力, 生成时关注约束 (实现要点见下文 6.3 节)
- **Constraint Prediction Head**
辅助预测当前生成实体应满足的约束

6.2 模型输入 / 输出

- **输入:** DeepCAD 命令序列 (与原版一致)
- **输出:** CAD 命令序列 + 约束标签 (辅助输出)

6.3 Constraint-Aware Cross-Attention 实现

6.3.1 作用与接口

标准交叉注意力: **Query** 为当前要建模的表示, **Key / Value** 为另一路信息。
Constraint-Aware Cross-Attention 指: 在生成 CAD 命令序列时, 解码器隐

状态作为 **Q**，约束 token 序列作为 **K、V**，使每一步表示能主动聚合「水平 / 竖直 / 平行 / 垂直 / 共线」及涉及线段等信息。

与 DeepCAD 解码器（自注意力 → 对 latent z 的交叉注意力或全局注入 → FFN）的关系：**在保留对编码器 memory 的前提下，增加一路对约束序列 c 的多头交叉注意力**，或先将 z 与 c 融合后再做一次交叉注意力（见 6.3.5）。

6.3.2 约束图序列化 (Constraint Tokenization)

离线约束是字典（单线列表 + 线对列表），不是自然序列。实现前须将 **每条约束实例** 变为 **定长向量序列**，才能接入 MultiheadAttention。

做法	说明
A. 一条约束 = 一个 token (推荐)	每个 token = 类型嵌入 + 实体嵌入（见下），与约束图一一对应，实现简单
B. 三元组 (head, rel, tail)	类似知识图谱，序列更长；对本任务五类约束通常不必

6.3.3 单条约束的向量 (Constraint Embedding)

对每条约束实例：

1. 类型嵌入

HORIZONTAL, VERTICAL, PARALLEL, PERPENDICULAR, COLLINEAR, padding 可用 NONE。

得到 $e_{\text{type}} \in \mathbb{R}^{d_{\text{model}}}$ （或与实体拼接后再线性投影到 d_{model} ）。

2. 实体嵌入

线段用 **序列内线索引** 或 **全局线 id** 查表：Embedding(N_{max}, d) 再可选 Linear $\rightarrow d_{\text{model}}$ 。

- **水平 / 竖直**：单线 id $\rightarrow e_{\text{line}}$ 。
- **平行 / 垂直 / 共线**：两线 id $\rightarrow e_a, e_b$ ，再 concat 后经 MLP，或 $e_a + e_b$ 等简单融合。

3. 合成约束 token

$h_c = \text{LayerNorm}(\text{Linear}([e_{\text{type}} ; f(e_{\text{lines}})]))$ # 或加权和再 LayerNorm

4. Batch 与 mask

堆成 $C \in \mathbb{R}^{B \times T_c \times d_{\text{model}}}$ ， T_c 为样本内约束条数（padding 至 batch 内最大），用 **key_padding_mask** 屏蔽 pad 位置。

6.3.4 交叉注意力计算形式

设解码器某层隐状态为 $H \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ (T 为命令序列长度)。

- $Q = H$ (可与现有实现一致: 先 LayerNorm 再注意力)
- $K, V = c$

即一次标准多头注意力后残差:

```
H' = MultiheadAttention(Q=H, K=C, V=C, key_padding_mask=constraint_pad)
H = H + Dropout(H')
```

与 PyTorch `nn.MultiheadAttention(tgt, memory, memory, key_padding_mask=...)` 形式相同, 仅第二路 `memory` 从编码器输出换为 ****约束 token 序列 c ****。

6.3.5 与 DeepCAD 解码器的衔接 (最小侵入)

DeepCAD 为 **Encoder $\rightarrow$ latent $z \rightarrow$ Decoder**。解码器内通常已有对 **memory (全局向量或序列)** 的交叉注意力, 或 GlobalImproved 式全局注入。

可选策略:

1. 双路交叉注意力 (优先实现)

每层顺序示例: Self-Attn $\rightarrow$ Cross-Attn(z) $\rightarrow$ Cross-Attn(C) $\rightarrow$ FFN。

或先将 z 与 c 在序列维拼接再线性压回 d_{model} 作为单一 `memory` (参数更多, 等价于更宽的 K/V)。

2. 仅在后若干层加入对 c 的 Cross-Attention

前几层行为与原版一致, 后 k 层再引入约束, 训练更稳。

3. 训练 / 推理时 c 的来源

- **训练**: 使用离线提取的 **GT 约束** token 化后与命令对齐。
- **推理**: 无约束时可 $\tau_c = \emptyset$ 或仅 NONE 占位; 若配合 **Constraint Prediction Head** 逐步预测约束再喂回, 难度更高, 可作扩展。

6.3.6 可选增强

- **与命令对齐**: 若线 id 对应「画线」命令在序列中的位置, 可将 **命令位置编码** 融入线嵌入, 强化「第几条线」与解码位置的一致性。
- **稀疏 attention**: 仅允许当前步关注与正在生成实体相关的约束, 需构造 `memory_mask`, 实现成本高; 初期可用全注意力 + padding mask。

- 与 $L_{\text{constraint}}$ 配合：Cross-Attention 负责表示中融入约束；**Constraint Prediction Head + 约束损失** 负责显式监督，二者互补。

6.3.7 实现 Checklist

1. 离线：约束字典 $\rightarrow$ batch 内 约束 token 序列（定长 pad）。
2. **Constraint Embedding**：类型 + 实体（+ 可选 MLP） $\rightarrow c$ 。
3. 解码器层中增加 ****MultiheadAttention(H, C, C, key_padding_mask)****（或与原有对 z 的 cross-attn 并列）。
4. 总损失： $L_{\text{total}} = L_{\text{cmd}} + \alpha * L_{\text{constraint}}$ （与第 7 节一致）。

7. 损失函数（总损失）

$$L_{\text{total}} = L_{\text{cmd}} + \alpha * L_{\text{constraint}}$$

L_{cmd} : 原生 DeepCAD 命令交叉熵损失

$L_{\text{constraint}}$: 约束分类 / 回归损失（平行 / 垂直 / 共线等）

$\alpha = 0.1 \sim 0.5$

8. 评估指标（可直接代码实现）

- 水平满足率 = 水平线段数 / 总线段数
- 竖直满足率 = 竖直线段数 / 总线段数
- 平行满足率 = 生成中满足的 GT 平行对数 / GT 平行总数
- 垂直满足率 = 生成中满足的 GT 垂直对数 / GT 垂直总数
- 共线满足率 = 生成中满足的 GT 共线对数 / GT 共线总数
- Chamfer Distance / Hausdorff Distance
- 拓扑合法率（闭合、无破面）

9. 代码文件结构（可直接交给 AI 生成）

```
constraint_deepcad/
├── data/
│   ├── constraint_extractor.py    # 约束自动提取
│   └── deepcad_dataset.py        # 继承原数据集，加载约束
├── model/
│   ├── constraint_embedding.py
│   ├── constraint_transformer.py
│   └── constraint_loss.py
├── evaluate/
│   └── metric_calculator.py      # 约束满足率计算
├── train.py
└── config.py
```

10. 最简实现路线 (AI 可直接执行)

1. 加载 DeepCAD 官方 JSON
2. 实现 ConstraintExtractor 类, 自动解析几何并输出约束
3. 改造 DeepCAD Transformer, 按 6.3 实现 Constraint Embedding 与 Constraint-Aware Cross-Attention
4. 实现 ConstraintLoss
5. 训练流程与原版对齐
6. 实现 ConstraintMetric 计算满足率

11. 不做什么 (避免 AI 跑偏)

- 不做 Pointer 机制、扩散模型、点云路线
- 不新增标注、不另建新数据集
- 不改 DeepCAD 命令表示格式
- 只做草图约束, 不做复杂 3D 特征约束

12. 可直接使用的论文式描述

本方法在不增加人工标注的前提下, 从原始 CAD 命令序列中自动挖掘几何约束, 通过约束感知的 Transformer 结构与约束一致性损失, 显著提升生成模型的约束满足率与工程合法性, 完全兼容 DeepCAD 数据集。